

Product Management & Order Processing System

Simple Project Documentation • Backend + Frontend • Interview / Assignment Reference

Project Purpose

This project is a small e-commerce style application for managing products, inventory and customer orders. It supports different user roles so that each type of user can access only the features they need. The main focus is correct role-based access, inventory validation and the complete checkout flow.

Technology Stack

Layer	Technology
Backend	Spring Boot
REST APIs	Spring Web / REST Controllers
Persistence	Spring Data JPA + Hibernate
Database	MySQL
Security	Spring Security + JWT
Validation	Jakarta Validation
Frontend	React
Frontend routing	React Router
API communication	Axios
UI	Material UI (MUI)

1. Main Features

- USER can view products by category.
- USER can add, update and remove items from the cart.
- USER can add and manage delivery addresses.
- USER can place an order only when required inventory is available.
- ADMIN can add and update products.
- ADMIN can manage product price and inventory.
- ADMIN can enable or disable products.
- ADMIN and SUPER_ADMIN can manage products and inventory.

- SUPER_ADMIN can manage categories, users and roles and can view all orders.

2. User Roles

Role	Main Responsibilities
USER	Browse products, view categories, manage cart, manage addresses, checkout and view own orders.
ADMIN	Product management, price management, inventory management and product enable/disable.
SUPER_ADMIN	All ADMIN permissions plus category management, user management, role management and viewing all orders.

3. Application Architecture

The backend follows a layered architecture. Controllers receive HTTP requests, services contain business logic, repositories communicate with the database, and entities represent database tables. DTOs are used for request and response data.

High-level flow

```
React Frontend → Axios → Spring Boot REST Controller → Service → Repository →  
Hibernate/JPA → MySQL
```

Security is applied around the API layer. The frontend sends the JWT in the Authorization header. The JWT filter validates the token and Spring Security uses the user's authorities to allow or deny protected requests.

4. Database Design

The main database tables are:

- users – stores application users.
- roles – stores USER, ADMIN and SUPER_ADMIN roles.
- user_roles – connects users with their roles.
- products – stores product information, price, category and active status.
- categories – stores product categories.
- inventory – stores available quantity for each product.
- cart – stores a user's cart.
- cart_items – stores products and quantities inside a cart.
- addresses – stores delivery addresses.
- orders – stores placed orders and order totals.
- order_items – stores the products and quantities belonging to an order.

Important relationships

A user can have roles, addresses, a cart and orders. A category can contain products. A product can have inventory and can appear in cart items and order items. An order contains multiple order items.

5. Authentication and JWT Security

The application uses username/password authentication with JWT. When the user logs in, Spring Security authenticates the credentials. A JWT is generated and returned to the frontend.

Request flow

Login → Authenticate credentials → Load user and roles → Generate JWT → Frontend stores token → Axios sends Bearer token → JWT filter validates token

The backend converts application roles such as ADMIN into Spring Security authorities such as ROLE_ADMIN. The backend remains the final authorization layer.

6. Role-Based Authorization

Role checks are configured in SecurityConfig. Public product viewing is allowed for GET product requests. Product and inventory management require ADMIN or SUPER_ADMIN. Category write operations, user management, role management and all-order access are restricted to SUPER_ADMIN.

Example permission model:

Feature	USER	ADMIN	SUPER_ADMIN
View products	Yes	Yes	Yes
Cart / Address / Checkout	Yes	Yes	Yes
Product management	No	Yes	Yes
Inventory management	No	Yes	Yes
Category write operations	No	No	Yes
User / Role management	No	No	Yes
View all orders	No	No	Yes

7. Product Management Flow

- SUPER_ADMIN creates categories.
- ADMIN creates a product and assigns a category.
- ADMIN can update the product, price and active status.
- ADMIN sets or updates inventory for the product.
- USER can then view active products and add them to the cart.

8. Cart Flow

A cart belongs to a user. A cart item stores the selected product and quantity. The user can add an item, change its quantity, remove an item or clear the cart.

9. Checkout and Order Processing

Checkout is the most important business flow. The system validates the cart before creating an order. It checks inventory for every cart item. If any requested quantity is greater than the available inventory, checkout fails.

Successful checkout flow

```
Cart → Validate cart → Check inventory → Create order → Create order items → Calculate total → Reduce inventory → Complete order
```

The inventory repository uses a pessimistic write lock for the inventory lookup used during stock-sensitive processing. This helps reduce the risk of two concurrent checkout requests updating the same stock incorrectly.

Checkout should be handled as a transaction because order creation, order-item creation and inventory changes belong to one business operation. If an important operation fails, the transaction can roll back instead of leaving partial data.

10. Exception Handling

The backend uses custom exceptions for common business errors and a global exception handler with `@RestControllerAdvice`. This keeps error responses consistent.

- `ResourceNotFoundException` – requested data was not found.
- `BadRequestException` – invalid business request.
- `DuplicateResourceException` – duplicate username, email or other unique data.
- `InsufficientInventoryException` – requested quantity is not available.
- `MethodArgumentNotValidException` – validation annotation failed.
- `DataIntegrityViolationException` – database constraint was violated.

11. DTO Usage

DTOs are used between the API and service layers instead of exposing JPA entities directly. This keeps the API contract separate from the database model and gives better control over what information is accepted and returned.

12. Frontend Structure

The React application uses a central Axios client. The client automatically reads the JWT token from local storage and sends it as a Bearer token with API requests. React Router handles navigation and `ProtectedRoute` provides frontend route protection.

The UI uses Material UI components and includes separate dashboard areas for USER, ADMIN and SUPER_ADMIN. The frontend role check improves navigation and user experience, while Spring Security on the backend remains the real security boundary.

13. Important API List

Area	Important APIs
Auth	POST /api/auth/register POST /api/auth/login

Area	Important APIs
Products	GET /api/products GET /api/products/category/{categoryId} POST /api/products PUT /api/products/{id} PATCH /api/products/{id}/enable PATCH /api/products/{id}/disable
Inventory	GET /api/inventory/{productId} PUT /api/inventory/{productId}
Cart	GET /api/cart POST /api/cart/items PUT /api/cart/items/{productId} DELETE /api/cart/items/{productId} DELETE /api/cart
Orders	POST /api/orders/checkout GET /api/orders/my-orders GET /api/orders/{orderId} GET /api/orders
Categories	GET /api/categories POST /api/categories PUT /api/categories/{id} DELETE /api/categories/{id}
Users	GET /api/users GET /api/users/{id} DELETE /api/users/{id}
Roles	PUT /api/roles/users/{userId}

14. Running the Project

Backend

The backend runs on port 8086.

```
Windows PowerShell:

cd D:\SpringBootProjects\Product-Order-System\product-order-system
.\mvnw.cmd spring-boot:run
```

Frontend

The React frontend runs on port 5173.

```
Windows PowerShell:

cd D:\SpringBootProjects\Product-Order-System\product-order-ui
npm run dev
```

The frontend is available at <http://localhost:5173> and the backend at <http://localhost:8086>.

15. Database Configuration

The application reads sensitive values such as the database password and JWT secret from environment variables rather than keeping them directly in the properties file.

```
Example configuration pattern:

spring.datasource.password=${DB_PASSWORD}
app.jwt.secret=${JWT_SECRET}
```

For a real submission, make sure actual passwords and JWT secrets are not committed to GitHub. Use environment variables or another secure configuration mechanism.

16. Test / Demo Flow for Interview

- Login as SUPER_ADMIN and create a category.
- Login as ADMIN and create a product under that category.
- Set inventory for the product.
- Login as USER and view the product.
- Add the product to the cart.
- Add/select an address.
- Checkout with a quantity that is available.
- Verify that the order is placed and inventory is reduced.
- Try checkout with more quantity than available and verify that checkout fails.
- Login as SUPER_ADMIN and demonstrate user, role, category and all-order access.

17. Interview Explanation – Short Version

“My project is a Product Management and Order Processing System, basically an e-commerce application. I used Spring Boot, Spring Data JPA, Hibernate, Spring Security and JWT on the backend, MySQL as the database, and React with Axios, React Router and Material UI on the frontend. The system has USER,

ADMIN and SUPER_ADMIN roles. The important business flow is checkout: before placing an order, I validate the cart and inventory. If stock is insufficient, checkout fails. If stock is available, the system creates the order and order items, calculates the total and reduces inventory. I also used DTOs, custom exceptions, global exception handling, transactions and pessimistic locking for inventory-sensitive processing.”

18. Key Technical Points to Remember

- JWT is used for stateless authentication.
- Spring Security enforces backend authorization.
- ProtectedRoute provides frontend route protection and better UX.
- DTOs keep API models separate from JPA entities.
- JPA/Hibernate maps Java entities to MySQL tables.
- Checkout validates inventory before placing an order.
- Inventory uses pessimistic locking for concurrent stock-sensitive access.
- Transactions help keep checkout changes consistent.
- GlobalExceptionHandler provides consistent error responses.
- Sensitive configuration should stay outside source control.

19. Project Status

The project is designed as an assignment/demo application. The main goal is correctness of the core product, inventory, cart, checkout and role-based flows rather than full production-scale infrastructure.

Prepared as a simple, human-readable project reference for interview and assignment submission.